



Systeme de mises a jour securise

Guide d'installation complet — v2

Serveur Node.js — Debian Linux

Site de telechargement integre :3000

Client Python — Windows (zero config)

Node.js 20 Python 3.10+ TLS 1.3 RSA 4096 SHA-256 HTTP :3000 HTTPS :8443

Table des matieres

1	Vue d'ensemble de l'architecture	3
2	SERVEUR — Installation sur Debian	4
2.1	Installation de Node.js et outils	4
2.2	Deployer les fichiers	4
2.3	Generer le certificat SSL	5
2.4	Generer les cle RSA et tokens	5
2.5	Creer les paquets de test	6
2.6	Demarrer les deux serveurs	6
2.7	Services systemd (demarrage auto)	7
3	SITE WEB — Page de telechargement	8
3.1	Comment ca fonctionne	8
3.2	Ce que contient le ZIP genere	8
3.3	Acceder au site	9
4	CLIENT Windows — Installation	10
4.1	Telecharger depuis le site	10
4.2	Installer Python	10
4.3	Installer les dependances	11
4.4	Lancer la mise a jour	11
4.5	Verification automatique	12
5	Scenarios de test de securite	13
6	Reference des fichiers	15
7	Resolution des problemes	16

1. Vue d'ensemble de l'architecture

Le systeme comprend desormais TROIS composants sur le meme serveur Debian : le serveur d'API securise (HTTPS :8443), le site web de telechargement (HTTP :3000), et le depot de paquets. Le client Windows n'a besoin que d'un navigateur pour tout recuperer.



Flux utilisateur simplifie

Etape	Action	Port
1	Le client ouvre son navigateur et visite http://IP:3000	HTTP :3000
2	Il clique sur 'Telecharger' — recoit update-client.zip	HTTP :3000
3	Il decompresse le ZIP (config.ini deja pre-rempli)	—
4	Il installe Python + pip install -r requirements.txt	—
5	Il lance python client.py	HTTPS :8443
6	Le client verifie le manifeste signe RSA	HTTPS :8443
7	Telechargement + verification SHA-256 + installation	HTTPS :8443

2. SERVEUR — Installation sur Debian

Node.js 20 Express HTTPS :8443 HTTP :3000

1 Installation de Node.js et outils

SSH Debian

```
# Mettre a jour le systeme
$ sudo apt-get update && sudo apt-get upgrade -y
# Installer Node.js 20
$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
$ sudo apt-get install -y nodejs openssl zip curl
# Verifier
$ node --version      # v20.x.x
$ npm --version       # 10.x.x
```

2 Deployer les fichiers

Transferez **serveur-debian.zip** sur le Debian :

Deploiement

```
# Depuis Windows PowerShell
> scp serveur-debian.zip USER@192.168.1.XXX:/home/USER/
# Sur le Debian
$ unzip serveur-debian.zip -d ~/update-system
$ cd ~/update-system/server
$ npm install
```

3 Generer le certificat SSL

Certificat SSL

```
$ cd ~/update-system/server
$ chmod +x setup-certs.sh
$ bash setup-certs.sh
IP locale detectee : 192.168.1.42
Certificat genere (valide 10 ans)
  certs/server.key  (NE PAS PARTAGER)
  certs/server.crt  (distribue automatiquement via le site)
```

i

Notez l'IP affichee — c'est l'adresse du site web que vous communiquerez a vos clients :
http://192.168.1.42:3000

4 Generer les cle RSA et tokens

Cles RSA + Tokens

```
$ node setup-keys.js
Cle privée -> keys/private.pem (NE PAS PARTAGER)
Cle publique -> keys/public.pem (distribuee via le site)

TOKENS A NOTER
Admin : a1b2c3d4... (acces aux logs)
Client : f6e5d4c3... (integre automatiquement dans config.ini du ZIP)
```



Le token client est integre automatiquement dans le **config.ini** du ZIP genere par le site. Le client n'a rien a configurer.

5 Créer les paquets de test

Paquets de test

```
$ chmod +x create-test-packages.sh
$ bash create-test-packages.sh
Paquet 1.0.0 cree
Paquet 1.0.1 cree
Paquet 1.0.2 cree <-- sera propose comme mise a jour
```

6 Demarrer les deux serveurs

Un seul script lance les deux serveurs simultanement :

Demarrage avec start-all.sh

```
$ chmod +x start-all.sh
$ bash start-all.sh

Verification des prerequis...
Cles RSA : OK
Certificat SSL : OK
IP locale : 192.168.1.42

API mises a jour : https://192.168.1.42:8443
Site telechargement: http://192.168.1.42:3000

Dites a vos clients de visiter http://192.168.1.42:3000
Ctrl+C pour tout arreter
```



Ou lancez chaque serveur separement dans deux terminaux :
node server.js (API) et **node download-site.js** (site)

7 Services systemd (demarrage automatique)

Pour demarrer automatiquement au boot, installez les deux services systemd :

Installation systemd

```
# Copiez les deux fichiers .service (editez d'abord votre nom d'utilisateur)
$ nano server/update-api.service # remplacez votre_utilisateur
$ nano server/update-site.service # remplacez votre_utilisateur

$ sudo cp server/update-api.service /etc/systemd/system/
$ sudo cp server/update-site.service /etc/systemd/system/
$ sudo systemctl daemon-reload
$ sudo systemctl enable update-api update-site
$ sudo systemctl start update-api update-site
$ sudo systemctl status update-api update-site

# Ouvrir les deux ports si pare-feu actif
$ sudo ufw allow 8443/tcp
$ sudo ufw allow 3000/tcp
$ sudo ufw reload
```

3. SITE WEB — Page de telechargement

download-site.js HTTP :3000 ZIP auto-genere

1 Comment ca fonctionne

Le site web genere le ZIP client **a la volee** a chaque telechargement, en y inserant automatiquement :

- L'IP du serveur detectee automatiquement
- Le token client lu depuis **tokens.json**
- Le certificat SSL **server.crt** depuis le dossier certs/
- La cle publique RSA **public.pem** depuis le dossier keys/
- Un **config.ini** entierement pre-rempli
- Les scripts **client.py**, **scheduler.py**, **requirements.txt**
- Un fichier **LIRE-MOI.txt** avec les instructions

v Le client recoit tout ce dont il a besoin dans un seul ZIP. **Il n'a rien a configurer manuellement.**

2 Ce que contient le ZIP genere

Contenu du ZIP telechargeable :

Fichier dans le ZIP	Contenu / Origine
update-client/client.py	Agent de mise a jour Python
update-client/scheduler.py	Verification automatique
update-client/requirements.txt	Dependances Python
update-client/config.ini	Pre-rempli : URL + token + chemins
update-client/server.crt	Copie de certs/server.crt du serveur
update-client/public.pem	Copie de keys/public.pem du serveur
update-client/LIRE-MOI.txt	Instructions en francais

Le config.ini genere ressemble a ceci :

[server]

url = https://192.168.1.42:8443

token = f6e5d4c3b2a1...

server_cert = server.crt

[security]

public_key_path = public.pem

3 Acceder au site

La page s'affiche dans n'importe quel navigateur sur le reseau local :

Acces au site

```
# Depuis un navigateur Windows sur le meme reseau :  
http://192.168.1.42:3000  
  
# La page affiche :  
- La derniere version disponible (ex: 1.0.2)  
- Le statut du serveur (En ligne / Hors ligne)  
- Un bouton Telecharger le client (ZIP)  
- Les 4 etapes d'installation a suivre  
  
# Si des fichiers manquent, la page affiche un avertissement :  
! Certificat SSL manquant -- lancez bash setup-certs.sh  
! Cle publique manquante -- lancez node setup-keys.js
```



Le bouton de telechargement est desactive automatiquement si le serveur n'est pas completement configure (cles ou certificat manquants).

4. CLIENT Windows — Installation

Python 3.10+ ZIP pre-configuré Zero configuration manuelle

1 Télécharger depuis le site

Sur le PC Windows, ouvrez un navigateur et visitez :

Navigateur Windows

```
http://192.168.1.42:3000  
(remplacez par l'IP affichée par start-all.sh)
```

Cliquez sur **Télécharger le client (ZIP)**. Enregistrez **update-client.zip** puis décompressez-le dans un dossier de votre choix (ex: **C:\update-client**).

✓ Le ZIP contient déjà config.ini, server.crt et public.pem pré-configurés. **Il n'y a rien à modifier.**

2 Installer Python (si nécessaire)

Si Python n'est pas installé sur le PC Windows :

PowerShell Windows

```
# 1. Téléchargez Python depuis https://python.org/downloads  
# IMPORTANT : cochez 'Add Python to PATH' à l'installation  
  
# 2. Vérifiez dans PowerShell :  
> python --version  
Python 3.11.x
```

3 Installer les dépendances

Dans PowerShell, allez dans le dossier décompressé et installez :

Installation des dépendances Python

```
> cd C:\update-client  
> pip install -r requirements.txt  
Successfully installed requests-2.31.0 cryptography-41.0.0
```

i Cette étape n'est nécessaire qu'**une seule fois**. Les mises à jour suivantes se font juste avec **python client.py**.

4 Lancer la mise à jour

Lancement de la mise a jour

```
> cd C:\update-client
> python client.py

Agent de mise a jour démarre
Vérification de la connectivité serveur...
Serveur joignable
Manifeste reçu : version 1.0.2
Signature du manifeste valide
Version actuelle : 0.0.0
Version disponible : 1.0.2
Téléchargement de update-1.0.2.zip...
Progression : 100.0%
Intégrité vérifiée (SHA-256 OK)
Sauvegarde créée
Version 1.0.2 installée
Mise à jour vers 1.0.2 terminée avec succès !
```

5 Vérification automatique (scheduler)

Pour vérifier automatiquement les mises à jour toutes les heures :

Vérification automatique

```
> python scheduler.py

Scheduler de mises à jour démarre
Intervalle : 3600 secondes (60 min)
Ctrl+C pour arrêter
```

i

Pour un lancement automatique au démarrage Windows : Planificateur de tâches → Nouvelle tâche → **python C:\update-client\scheduler.py** au démarrage de session.

5. Scenarios de test de securite

7 scenarios pour valider chaque mecanisme

Executez ces tests dans l'ordre pour valider le systeme complet.

Test 1 — Telechargement depuis le site

Action	Visitez <code>http://IP:3000</code> depuis Windows, cliquez Telecharger
Attendu	ZIP reçu avec <code>config.ini</code> , <code>server.crt</code> , <code>public.pem</code> pre-remplis
Verif	Ouvrez <code>config.ini</code> : l'IP et le token doivent etre corrects

Test 2 — Mise a jour normale

Condition	Client a la version 0.0.0, serveur propose 1.0.2
Action	<code>python client.py</code>
Attendu	Telechargement + installation + <code>state.json</code> mis a jour

Test 3 — Deja a jour

Condition	Relancez <code>client.py</code> apres le test 2
Attendu	Deja a jour. Aucune action necessaire.

Test 4 — Serveur inaccessible

Condition	Arretez le serveur (Ctrl+C sur le Debian)
Attendu	Serveur inaccessible. Arret. — aucune modification locale

Test 5 — Token invalide

Condition	Modifiez token dans <code>config.ini</code> avec n'importe quoi
Attendu	Erreur HTTP 403 — Token invalide. Rien de telecharge.

Test 6 — Signature corrompue

Condition	Sur le Debian : modifiez le champ signature dans packages/1.0.2/meta.json
Attendu	ALERTE : Signature invalide ! Abandon. — aucune installation

Test 7 — Hash corrompu

Condition	Dans meta.json, remplacez sha256 par 000000...
Attendu	ALERTE : Hash invalide ! — fichier temporaire supprime

6. Reference des fichiers

Cote serveur (Debian)

Fichier	Role
<code>server/server.js</code>	Serveur API principal (HTTPS :8443)
<code>server/download-site.js</code>	Site web de telechargement (HTTP :3000)
<code>server/start-all.sh</code>	Lance les deux serveurs en meme temps
<code>server/setup-keys.js</code>	Genere les cle RSA + tokens
<code>server/setup-certs.sh</code>	Genere le certificat SSL
<code>server/create-package.js</code>	Ajoute un nouveau paquet de mise a jour
<code>server/update-api.service</code>	Service systemd pour l'API
<code>server/update-site.service</code>	Service systemd pour le site web
<code>server/tokens.json</code>	Tokens d'authentification (genere)
<code>server/audit.log</code>	Journal de toutes les actions
<code>keys/private.pem</code>	Cle privee RSA — NE PAS PARTAGER
<code>keys/public.pem</code>	Cle publique — distribuee via le ZIP
<code>certs/server.crt</code>	Certificat SSL — distribue via le ZIP
<code>certs/server.key</code>	Cle SSL — NE PAS PARTAGER
<code>packages/X.Y.Z/</code>	Dossier d'une version (meta.json + fichier)

Cote client Windows (dans le ZIP telecharge)

Fichier	Role
<code>client.py</code>	Agent principal — lance les mises a jour
<code>scheduler.py</code>	Verification automatique (toutes les heures)
<code>config.ini</code>	DEJA PRE-REMPLI — URL + token + chemins
<code>requirements.txt</code>	Dependances Python a installer une fois
<code>server.crt</code>	Certificat SSL du serveur (copie auto)
<code>public.pem</code>	Cle publique RSA (copie auto)
<code>LIRE-MOI.txt</code>	Instructions en francais
<code>state.json</code>	Version installee + historique (genere)
<code>client.log</code>	Journal des operations (genere)

backups/	Sauvegardes avant chaque mise a jour
app/	Dossier d'installation de l'application

7. Resolution des problemes

x Le site web n'est pas accessible (http://IP:3000)

- Verifiez que download-site.js tourne : ps aux | grep node
- Verifiez le port : sudo ss -tlnp | grep 3000
- Ouvrez le port si pare-feu : sudo ufw allow 3000/tcp
- Testez depuis le Debian : curl http://localhost:3000

! Le bouton Telecharger est grise

- Lancez d'abord node setup-keys.js (tokens + cles RSA)
- Lancez bash setup-certs.sh (certificat SSL)
- Rafraichissez la page apres ces etapes

! Le ZIP telecharge est corrompu

- Verifiez les logs du site : tail -f logs/site.log
- Verifiez que client.py existe bien dans le dossier client/
- Verifiez les droits : ls -la keys/ certs/ (doivent etre lisibles)

! Erreur SSL cote client apres extraction du ZIP

- Verifiez que server.crt est dans le meme dossier que client.py
- Le chemin dans config.ini doit etre : server_cert = server.crt
- Mode test : mettez server_cert = false pour desactiver la verif SSL

x Erreur 401/403 apres extraction

- Le token dans config.ini est incorrect (genere avant setup-keys.js ?)
- Supprimez le ZIP, relancez start-all.sh, retelechargez
- Verifiez tokens.json sur le serveur

i python client.py : ModuleNotFoundError

- pip install -r requirements.txt (depuis le dossier update-client)
- Verifiez Python : python --version (besoin 3.10+)
- Si plusieurs Python : py -3 -m pip install -r requirements.txt

Commandes de diagnostic

Diagnostic

```
# Verifier que les deux serveurs tournent
$ ps aux | grep node

# Voir les logs en temps reel
$ tail -f ~/update-system/logs/api.log
$ tail -f ~/update-system/logs/site.log

# Tester les deux serveurs depuis le Debian
$ curl http://localhost:3000
$ curl -k https://localhost:8443/health

# Tester la connexion depuis Windows (PowerShell)
> Test-NetConnection -ComputerName 192.168.1.42 -Port 3000
> Test-NetConnection -ComputerName 192.168.1.42 -Port 8443

# Voir les logs du client Windows
> type C:\update-client\client.log
```